

Sprite Generation Using CollaGAN and Diffusion Model

Project Report : CS 7643

Anjola Olarinde*, Siddharth Kapoor*, Jawad Khattak*, Scott Seago*
Georgia Institute of Technology

aolarinde3@gatech.edu, skapoor89@gatech.edu, jkhattak3@gatech.edu, sseago3@gatech.edu

Code: <https://github.gatech.edu/SpriteSquad/SpriteGen>

Abstract

Producing full sprite sheets is time intensive because artists must draw multiple poses for each character. Recent work treats this as missing data imputation problem. Given some observed poses, a model infers the missing ones using input-dropout schedules. We reproduce the MDI-GAN (CollaGAN-based) method of Coutinho & Chaimowicz (2024) in PyTorch and confirm the study’s main finding, performance declines as fewer views are available, and dropout schedules significantly improves robustness. With ASHA Hyperband tuning and larger batches, our best model exceeds the author’s strongest baseline. We additionally adapted a modality-masked diffusion model (inspired by M2DN) to the same task. Diffusion achieved competitive L1 performance and outperformed CollaGAN in the most challenging one source setting, but showed higher FID in easier cases and produced overly smooth, less consistent sprites. Overall dropout scheduling emerges as an effective principle for both GAN and diffusion models in missing pose sprite imputation.

1. Introduction/Background/Motivation

The goal of this project is to automatically generate missing characters’ sprites poses by treating the task as a missing-data imputation problem rather than a general image-generation task. Instead of creating entire sprites, we attempt to infer a missing pose from the poses that already exist. This following the approach introduced by Coutinho & Chaimowicz (2024) [3] who showed that a multi-input CollaGAN can successfully reconstruct a missing pose even when one, two, or three inputs are missing. Their key idea is that controlled input dropout schedules improves learning under missing view scenarios. Our objective was to replicate the behavior of their model, specifically the architecture and dropout based training strategy and evaluate whether the same trends hold when implemented in

PyTorch. As an exploratory extension, we also adapted a modality-masked diffusion model to test whether dropout schedules could similarly benefit diffusion based imputation. This is a change from our original project proposal, as instead of comparing CollaGAN results to the same prior baselines as done in Coutinho & Chaimowicz (2024) [3], we compared CollaGAN results to those from our new experimental diffusion model.

Today, most 2D game sprites are created manually; artists draw each frame by hand using illustration tools such as Photoshop to adjust pixels, colors, and character proportions for each image in the animation sequence [1, 2, 14]. A single sprite effect requires multiple stages, including concept design, frame-by-frame drawing, coloring, retouching, and revision which can take several days up to two weeks depending on complexity and team feedback cycles [15]. Because many games often require dozens to hundreds of different sprites and each sprite may include 10 to 30 sequential images, this manual workflow imposes a significant burden on artists and increases development cost [15]. Although machine learning methods have been explored to help automate sprite creation, existing approaches remain limited. Prior GAN-based methods often target narrow tasks such as recoloring, stylist variation, or generating simple character sprites from small proprietary datasets [7, 6, 15]. These models struggle to generalize to new sprite effects, and they frequently fail to preserve the structural integrity required for usable game assets. As a result, they cannot replace the manual drawing, evaluation, and correction steps that artists must still perform. Because of these limitations, sprite production remains slow, expensive, and difficult to scale despite ongoing interest in procedural content generation [16].

Automating sprite creation matters because it reduces one of the most time-consuming and repetitive tasks in 2D game development. Instead of redrawing every pose from scratch, artists could generate missing views automatically and refine only what needs adjustment. This would shorten production cycles, allowing faster iteration, and lower the cost of producing animated characters. Successful automa-

*Random order. All authors made equal contribution.

tion would especially benefit small studios and indie developers by lowering the barrier to producing high quality animated characters. More broadly, it also advances research on generative models for low-resolution, stylized data, potentially enabling new tools that make asset creation faster, cheaper, and more accessible.

In this study, we used the Pixel Art Character (PAC) dataset introduced by Coutinho & Chaimowicz (2024) [3]. The full dataset contains 14,202 paired samples of 2D game characters, where each sample includes four canonical poses: right, left, front, and back. These images span a wide range of pixel-art styles and character types, including humanoids, animals, monsters, and vehicles. Because every character is represented using the same four view structure, the dataset is particularly well suited for evaluating missing pose imputation. The original images varied in size across sources, so the author standardized them by padding or resizing each sprite to a uniform 64×64 resolution and adds an alpha channel for transparency [3]. This reprocessing ensures consistent tensor dimensions, enabling stable training and allowing convolutional generative architectures such as CollaGAN to operate reliably. Following the original study [3], we used the same 85%-15% train-test split, consisting of 12,074 training images and 2,128 test images, and applied the same strategies described in the author’s implementation.

2. Approach

To replicate the study Coutinho & Chaimowicz (2024) [3], we re-implemented the MDIGAN architecture in PyTorch and built a unified PyTorch Lightning pipeline that supports both GAN based and Diffusion based missing-data imputation. Within this framework, we used Optuna for hyperparameter tuning and TensorBoard for real-time loss visualization and qualitative inspection of generated sprites. We implemented a custom Data Module (*MDIDataModule*) to load the MDI PAC dataset into a unified tensor format [*Batch Size, 4, Channels, Height, Width*], where each of the four entries corresponds to a sprite pose. We used the same reprocessing steps, normalizing, alpha-channel handling, and hue rotation augmentation, as implemented in the author’s official MDIGAN GitHub Repository.

Instead of removing poses at the dataset level, we followed the original MDIGAN training methodology and simulated missing domains use the author’s dropout schedules. A dropout mask determines which poses are available as input; dropped poses are replaced with zeros and the generator receives a target-domain label so that it learns to ignore zeroed inputs [3]. This mechanism is the core contribution of the original work, and reproducing it correctly was essential for replicating their results. Our implementation faithfully reproduces the three dropout strategies described in the paper: *none, conservative, balanced, and*

curriculum, allowing us to test how each affects imputation performance.

For GAN training, we used the same loss components as the authors: adversarial loss, domain classification loss, L1 reconstruction loss, backward cycle consistency, and SSIM based structural similarity [3]. Because GANs are highly sensitive to hyperparameters, we performed multi-trial hyperparameter tuning using Optuna’s ASHA (hyperband) scheduler which terminates under-performing trials early. This was important because training can quickly diverge if, for instance, the discriminator makes a sudden leap in performance but the generator does not. Several hyperparameter configurations that appeared stable during short warm-up runs diverged when trained to the full 240k steps. However with ASHA, we consistently identified models that generalized better and ultimately produced models that outperformed the authors’ Schedule-1 baseline.

As a baseline validation step, we verified that our PyTorch implementation reproduced the qualitative trends reported in the original MDIGAN paper [3]. Especially that dropout schedules significantly improve reconstruction when input poses are missing and that performance degrades smoothly as fewer views are provided.

Beyond replication, our main novel contribution was the adaptation of the M2DN modality-masked diffusion model to the sprite imputation setting [11]. While MDIGAN uses dropout and zeroed inputs to represent missing domain, M2DN treats missing modalities as noised channel with the forward diffusion process [11]. We adapted this paradigm by applying controlled Gaussian noise to missing poses and conditioning the model on a mask indicating which domains were available. This allowed us to directly compare whether dropout schedules, originally proposed for GANS, also benefit diffusion based imputation. To ensure comparable evaluation, we trained both models on the same dataset, with identical splits, pre-processing, and mask structures, and evaluated them using L1 error, FID, and qualitative visual inspection.

We anticipated that GANS could be unstable to train, since the Generator and Discriminator can easily fall out of balance if one learns faster the other. We also expected sensitivity to hyperparameter (e.g., learning rate and beta values), and implementation challenges related to handling a variable number of available poses as input to CollaGAN.

For the diffusion model, we anticipated that the iterative denoising process would be slow, requiring hundreds of steps per generated image in the reverse diffusion process. We also expected high computational and memory costs from training a U-net with multiple skip connections and attention layers. Additionally, unlike the CollaGAN paper, the M2DN model does not have a public implementation, meaning we would need to recreate the forward and

reverse diffusion processes from scratch.

In a departure from the proposed model in Ho et al (2020) [5], rather than applying gaussian noise during training to the full input, forward diffusion is only applied to the missing poses, leaving the other inputs intact. During the reverse process, both missing (noised) and present images are reconstructed. This design is necessary because M2DN performs joint reconstruction; present modalities self reconstruct, while missing ones are synthesized from context. In addition, the model predicts the original input images rather than the applied noise.

We did not copy or extend any existing public implementation. Instead, we re-implemented the original TensorFlow MDIGAN architecture [3] entirely in PyTorch, including the input dropout mechanism and multi-branch generator design. For the diffusion model, no public code was available for M2DN [11], so we implemented the full diffusion pipeline ourselves, including the forward diffusion schedule, modified U-NET, and conditioning mechanism for missing pose synthesis. Our PyTorch Lighting pipeline, data loading structure, and training loops were all written from scratch to support both GAN based and diffusion based models under a unified framework. The diffusion model implementation was based on the implementation discussed in "The Annotated Diffusion Model" blog post[12], which was in turn based on denoising-diffusion-pytorch [13]. The base diffusion implementation was then modified according to the architecture described in Meng et al.[11]

3. Experiment Architecture

The data points in the PAC dataset containing 4 poses of a sprite $\mathbf{x} = (x_{back}, x_{left}, x_{front}, x_{right})$, where each pose is a 64×64 RGBA image $x_d \in \mathbb{R}^{64 \times 64 \times 4}$, where the posture aligns with a domain $d \in \{0, 1, 2, 3\}$. Then while we train, we sample the domain index d_t and ask the model to reconstruct the ground truth image, and draw a dropout mask to indicate which of the remaining domains the model can use. The model's input is namely $\tilde{\mathbf{x}} = \mathbf{m} \odot \mathbf{x}$ along with the target domain index d_t (for GAN) or the full mask $\mathbf{m}$ (for Diffusion), where the model needs to generate the x_{d_t} from the other poses.

Generator

The model architecture we use follows the CollaGAN based design introduced by Lee et al. (2019) and later adapted for pixel-art sprite generation by Coutinho & Chaimowicz (2024). The generator receives four encoder branches, one for each available input pose, and each branch receives both the source image and the spatially replicated one-hot encoder vector indicating the target pose. Each encoder contains four down sampling blocks, and the resulting feature maps are concatenated at a shared bottleneck. A single decoder with four upsampling blocks then reconstructs the tar-

get pose, with skip connections linking encoder activation to the corresponding decoder layers.

Compared to the original CollaGAN, the adapted model uses $64 \times 64 \times 4$ sprites instead of $128 \times 128 \times 3$ images and increases the number of channels to improve capacity [3, 10]. All generator components contain learnable parameters, including the convolutional layers, encoder and decoder branches, and shared bottleneck. Training uses the same composite loss formulation described in the original work; Adversarial loss to ensure realism, domain classification loss to enforce pose-specific structure, L1 reconstruction loss between generated target and ground truth target, backward cycle consistency L1 loss, an SSIM based structural similarity term. Together, these losses balance pixel accuracy, pose reconcilability and perceptual realism.

Discriminator

The discriminator follows the same design as in the original CollaGAN model [10]. It receives a batch of images and outputs two quantities: D_{adv} is an adversarial score indicating whether an image is real or generated and D_{dmn} is a domain classification probability distribution over the four pose categories (back, front, left, and right). Architecturally, the discriminator consists of six down-sampling blocks, each made of a convolution with stride 2 that halves the spatial resolution while increasing the channel depth, followed by a Leaky ReLU activation. The final down-sampling block includes an additional dropout layer to stabilize training. After shared layers, the network splits into two parallel convolutional heads where one is producing the adversarial output D_{adv} with a linear activation, and the other producing the domain classification output D_{dmn} with a softmax activation function [3]. The discriminator only has learned parameters in its convolutional layer and two convolutional heads and trained with Least-Squared GAN loss combined with cross-entropy loss on the domain classification head.

3.1. Diffusion Model Architecture

In addition to the GAN based CollaGAN architecture, we introduced a diffusion based model inspired by the *Multi-Modal Modality-Masked Diffusion Network for Brain MRI Synthesis With Random Modality Missing* for image synthesis [11] to explore whether denoising-based generative methods can support missing pose sprite reconstruction. The paper's model expresses different MRI pictures (i.e., T1, T1c, T2, FLAIR) as a different modality, so our model considers 4 sprite poses (i.e., front, back, left, right) as each modality. The model accepts up to four input poses, each processed through a lightweight encoder branch. These encoded features are concatenated along with a spatially broadcast one-hot pose mask that specifies the target poses to be generated. A learned diffusion time embedding is also

injected into the network to condition the denoising process at each timestep as shown in Figure 1.

Input and Encoding

The core of the model follows a modified U-Net style encoder and decoder architecture, where all weights are learned. Before the U-Net encoder, each input image domain has a dedicated convolution block, including group normalization and downsampling. The U-Net encoder consists of a sequence of residual blocks with self-attention and down sampling layer which progressively reduce spatial resolution while increasing channel depth to capture global pose structure. The time embedding is added at each stage of the encoder to inform the network of the current denoising step, allowing the model to behave differently in each (high-noise) versus late stages (low-noise). The residual block stabilize training and expand representational capacity, while attention layers help the network model long-range relationship that are important for preserving sprite shape and silhouette. At the bottleneck additional convolution layers refine the combined representation by emphasizing the most informative feature channels before decoding. The diffusion beta schedule as well as the forward diffusion operator are derived analytically from formulas and are not learned because they impose the stochastic process that the model learns to denoise in decoding. Unique to the M2DN architecture is the addition of channel attention layers at the final encoder and initial decoder stage [8] as well as a binary modality/domain mask which is concatenated with the feature channels at the end of the bottleneck stage. The modality mask provides feature layers in the model which allow the network to treat image generation from noise for missing channels differently than it treats image self-generation from intact input poses.

Decoder and Output

The decoder mirrors the encoder with up-sampling layers, residual blocks, and skip connections that restore fine spatial detail. Four shallow output branches map the decoded features to the four canonical poses, and the output corresponding to the target pose is selected. This architecture enables the diffusion model to reconstruct the missing view from the subset of available poses, treating missing inputs as noise and learning a unified latent representation conditioned to the pose mask. The model is trained with a mean squared error that tracks the difference between the generated sprite and the ground truth with the training objective to learn a denoising function that can reconstruct the ground truth sprite from a noised-image.

3.2. Hyperparameters

Both models expose a myriad hyperparameters which control how well the models will accomplish their task and for some, can be the determining factor as to whether

the model converges. For CollaGAN, the learning rate (Adam step size η) and beta parameters for Adam momentum learners (in particular the first-moment coefficient β_1) are particularly important because GANs are particularly noisy and affected by faulty gradients. Having a strong, but not dominant learning rate is vital for the discriminator and generator not becoming overly confident, but also taking sizeable steps towards moving through the parameter space. λ_{SSIM} controls how much the model values structure, where a high value can preserve limbs and motif structure and a low value only values lower L1 loss. Similarly, the other loss weights λ_{L1} , λ_{L1_back} and λ_{domain} balance pixel accuracy, cycle consistency, and correct pose classification in the generator’s composite loss. The final set of hyperparameters for CollaGAN is shown in Table 4 in the Appendix.

For Diffusion, the timesteps the model takes (denoted T) directly impacts how much computational time the model needs to denoise the picture, and control the trade-off between sample quality and speed. Our team had limited computational power and resources, but we understood that availability of resources impacts the model’s ability to produce a quality image and would impact our assessment of how Diffusion compared to CollaGAN. We initialized hyperparameters using the papers’ inputs (for example learning rate=0.0005, $\beta_1 = 0.9$, total_steps=240000, cosine beta schedule with offset $s = 0.008$, and we set batch size to 4. We then used Optuna to tune hyperparameters using a small validation set using L1 for CollaGAN and adversarial MSE loss for Diffusion while monitoring FID scores. The final set of hyperparameters for M2DN is shown in Table 3 in the Appendix.

4. Results

To evaluate whether our replication successfully reproduced the behavior of the original CollaGAN approach, we used the same quantitative metrics reported by Coutinho & Chaimowicz (2024) [3]. We evaluated the model using L1 reconstruction error and Fréchet Inception Distance (FID), and complimented these quantitative metrics with qualitative visual inspection of the generated images. The L1 reconstruction error measures the absolute pixel-wise difference between a generated image and the corresponding ground-truth image [9]. We used the FID to quantify perceptual realism by comparing the statistical distance between generated and real image feature distribution [4]. In addition to quantitative metrics, we conducted qualitative visual evaluation, a standard practice in GAN research for assessing realism and structural consistency of generated samples. We conducted three main experimental conditions reflecting different levels of missing information including one missing pose (3 inputs $\rightarrow$ 1 output), two missing poses (2 inputs $\rightarrow$ 2 outputs), and three missing pose (1 inputs $\rightarrow$ 3 outputs). These progressively harder conditions allowed us to examine how well CollaGAN generalized when less

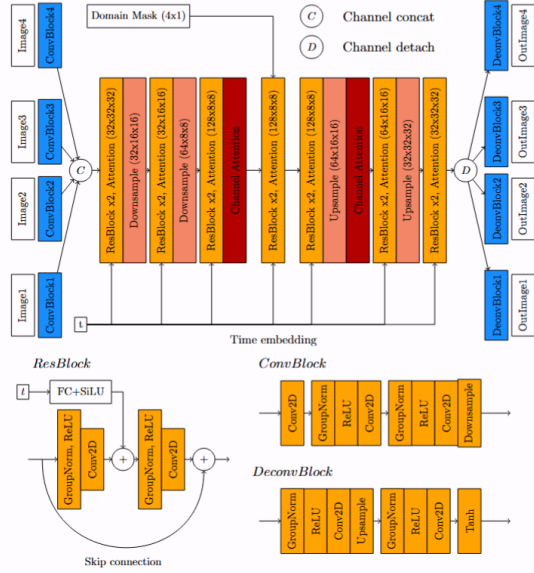


Figure 1: Architecture of Diffusion Model based on Multi-Modal Modality-Masked Diffusion Network (M2DN) [11]

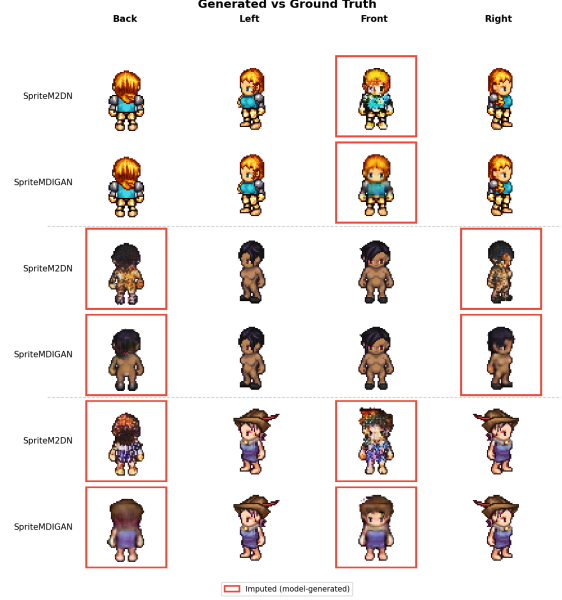


Figure 2: Image generation comparison for both models. The first example is for a one-missing scenario, and the others are for two-missing scenarios

Metric	Sources	SpriteMDIGAN				SpriteM2DN			
		None	Conservative	Curriculum	Balanced	None	Conservative	Curriculum	Balanced
FID	3	71.00	67.71	61.42	51.73	88.61	79.18	74.98	75.56
FID	2	99.61	78.01	78.72	71.56	117.81	82.86	76.31	80.34
FID	1	152.42	104.91	112.48	100.22	196.77	88.37	80.28	86.18
L_1	3	0.0556	0.0515	0.0630	0.0645	0.0604	0.0621	0.0622	0.0662
L_1	2	0.1001	0.0574	0.0651	0.0682	0.1188	0.0706	0.0692	0.0732
L_1	1	0.1516	0.0691	0.0733	0.0751	0.3003	0.0810	0.0781	0.0831

Table 1: FID and L_1 metrics for SpriteMDIGAN and SpriteM2DN under different dropout strategies.

structural information is available. Generated images from both models can be compared in Figure 2, with additional visualizations in the Appendix in Figure 4, Figure 5, and Figure 6.

4.1. CollaGANs Result

We trained the CollaGAN using the same pre-processing, augmentation, and train-test split as the original paper Coutinho & Chaimowicz (2024) [3]. For each experiment, we measure L1 loss averaged across the test set, FID between generated poses and real poses and visual quality based on qualitative inspection such as sharpness, structure preservation, color consistency, and pose accuracy.

Our CollaGAN replication exhibited the same qualitative behaviors reported in the original MDIGAN study, demonstrating that the model reliably imputes missing sprite poses and benefits substantially from structured input dropout schedules as shown in Figure 2. Quantitatively, we evaluated performance using L1 reconstruction error and Fréchet Inception Distance (FID) across three difficult setting; three

available poses, two available poses, and one available pose. Consistent with prior work, reconstruction accuracy degraded as fewer input poses were available, but dropout strategies markedly improved results relative to the **None** schedule. For example, with only one available pose, the L1 error under **None** was **00.1516**, whereas all dropout schedules more than **halved this error (0.073 - 0.076)**, with **Curriculum** providing the best overall reconstruction of **0.0733**. FID allowed the same pattern where **Balanced** dropout achieved the lowest FID across nearly all condition. For instance, **51.73** for three sources compared to **71.00** for No dropout which indicates that dropout trained models produced images that were both more realistic and more stylistically coherent with the dataset. The best FID score were observed under **Balanced** dropout schedule as shown in Table 1 compared to dropout types. This could be due to how it exposes the model to a mix of easy and difficult training cases, preventing overfitting to any single difficulty level. This diversity in input patterns help the gener-

ator learn more stable cross domain mappings, which is reflected in consistently lower FID score. Importantly, our L1 values were comparable to, or slightly better in some cases, compared to the original paper’s result as shown in Table 1. This could be attributed to improved hardware that enables a larger batch size (8 instead of 4) and ASHA-based hyperparameter tuning that identified more stable long-horizon training configuration.

On the other hand, our FID values were consistently higher than those reported by the authors. Because qualitative inspection showed that our generated sprites closely matched the samples in the original paper and demo site, we believe this discrepancy is due to implementation difference in FID computation (PyTorch-FID vs. TensorFlow’s internal FID), as well as heterogeneity of the PAC dataset, where a small set of stylistic outliers may inflate distribution based metrics.

The training dynamics further support this interpretation, as shown in appendix Figure 7. The **Curriculum** run exhibits pronounced mid training instability between roughly **60-120k steps**., followed by a steady improvement phase. All schedules ultimately trend downward, with the ASHA selected configuration converging to a lower final L1 than it began with. Notably, the model continues improving past **160k steps** and has not yet plateaued at **240k steps**, suggesting that additional training time or exploring dropout schedules with larger batch sizes could yield further gains. Overall, our results confirm that input dropout remains a powerful mechanism for missing pose imputations and our SpriteMDIGAN replication faithfully captures the core behaviors and finding of the original MDIGAN architecture.

4.2. Diffusion Model Results

We also evaluated our diffusion based model using the same three missing pose conditions and four dropout strategies from the CollaGAN framework (*None, Conservative, Curriculum, and Balanced*). The output from the model is shown in Table 1. Across all conditions, Curriculum strategy yielded the lowest FID score for all input domains. This could be due to diffusion model being exposed to progressively harder missing view configuration, allowing it to first stabilize structural priors from easier cases before generalizing to more ambiguous ones. Diffusion models are highly sensitive to condition quality, so beginning with complete information helps the network learn a coherent latent space and reduce instability. As a result, Curriculum avoids the structural drift and over smoothing in the other strategies, leading to lower FID across all experimental condition. Conversely, the None strategy had greater, but similar FID and L1 scores to the Conservative, Curriculum, and Balanced strategies across the 3-source input domain according to Table 1, but suffered for the 2-source domain and even more so for the single-source input. While this suggests that models without explicit drop-out training will

be unable to properly handle missing data at inference time, it also shows that models with drop-out training, especially when structured in a Curriculum-like strategy, force the network to learn view-invariant representations and not rely on pixel correlations. Overall image quality was poorer for our diffusion model than it was for CollaGAN. This is shown in the measured FID and L_1 metrics as well as qualitatively in the included image generation samples. This is partially a reflection of the limited time available to develop the architecture, but it may also reflect an inherent limitation of the original DDPM architecture when applied to this problem space.

5. Limitations and Challenges

Our results were heavily impacted by resource limits, which hinders the strength of both CollaGAN and Diffusion models. Training diffusion with a batch size of 4 required access to GPUs with at least 80GB of RAM. Testing was performed using an Nvidia H100 with 96GB of VRAM. With this hardware available, both models were trained with a batch size of 4. Even with more powerful GPUs and a small batch size of 4, a full 240,000 step run takes 6 hours. With CollaGAN, performing a sample run as a test, we estimate that a full run using a batch size of 16 would take 36 hours, where we unfortunately didn’t have the time or resources to devote. Additionally, the CollaGAN paper’s accompanying codebase shared specific implementation and hyperparameters, but the M2DN paper did not share code or all hyperparameters, having us devote more time to implementation from the paper and tuning, rather than just fine-tuning from the existing codebase to our modalities.

6. Conclusions

Our model was successfully able to build from the original authors’ MDIGAN architecture and replicate their results, confirming their claim that input-dropout schedules are vital for building robust models that can handle missing data. Then, using ASHA hyperparameter tuning and a larger batch size, SpriteGAN was able to improve the baseline, further validating their results and cementing the need for strong parameters. Diffusion did achieve competitive L1 scores on the most difficult tasks (single source inputs), it still suffered greater blurriness and higher FID compared to CollaGAN, likely because of time and computational resources. But though diffusion experiments reported higher FID scores, dropout schedules were shown to be of model-agnostic benefit, showing the choice of dropout schedule may be as important as the choice of the overall model.

7. Future Work

Another area to explore around diffusion models would be to create a model based on more recent latent diffusion model architectures. Applying diffusion to a shared latent space rather than the pixel-space based methods of *Denoising Diffusion Probabilistic Models* could provide much better rendition of missing poses.

Student Name	Contributed Aspects	Details
Scott Seago	Diffusion Implementation	Adapted the M2DN architecture to sprites in PyTorch, built the masked diffusion UNet, trained model for diffusion, tuned hyperparameters, and summarized diffusion experiments.
Siddharth Kapoor	CollaGAN	Implemented CollaGAN and losses in PyTorch, configured training/tuning (e.g., loss weights), ran CollaGAN experiments, and gathered FID results.
Jawad Khattak	Analysis & Validation	Built the initial TensorFlow prototype and data handling, ran experiment checks, analyzed experimentation results, and contributed to comparisons between implementations.
Anjola Olarinde	Diffusion Logic	Worked the forward/reverse DDPM and diffusion logic to align with paper’s diffusion process, fixed evaluation/masking issues, and helped compile written report.

Table 2: Contributions of team members.

References

- [1] Saiqa Aleem, Luiz Fernando Capretz, and Faheem Ahmed. Game development software engineering process life cycle: a systematic review. *Journal of Software Engineering Research and Development*, 4(1):6:1–6:30, 2016. 1
- [2] Marcos Silvano Orita Almeida and Flavio Soares Corrêa da Silva. A systematic review of game design methods and tools. In *International Conference on Entertainment Computing*, pages 17–29. Springer, 2013. 1
- [3] Flávio Coutinho and Luiz Chaimowicz. A missing data imputation GAN for character sprite generation. *arXiv preprint arXiv:2409.10721*, 2024. 1, 2, 3, 4, 5, 8
- [4] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30, 2017. 4
- [5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. <https://arxiv.org/abs/2006.11239>. 3
- [6] Seungjin Hong, Sookyun Kim, and Shinjin Kang. Game sprite generator using a multi discriminator gan. *KSI Transactions on Internet and Information Systems*, 13(8):4255–4270, 2019. 1
- [7] Lewis Horsley and Diego Perez-Liebana. Building an automatic sprite generator with deep convolutional generative adversarial networks. In *2017 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 134–141. IEEE, 2017. 1
- [8] Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu. Squeeze-and-excitation networks, 2019. <https://arxiv.org/abs/1709.01507>. 4
- [9] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017. 4
- [10] Dongwook Lee, Junyoung Kim, Won-Jin Moon, and Jong Chul Ye. Collagan: Collaborative GAN for missing image data imputation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019. 3
- [11] Xiangxi Meng, Kaicong Sun, Jun Xu, Xuming He, and Ding-gang Shen. Multi-modal modality-masked diffusion network for brain mri synthesis with random modality missing. *IEEE Transactions on Medical Imaging*, 43(7):2587–2598, 2024. 2, 3, 5
- [12] Niels Rogge and Kashif Rasul. The annotated diffusion model. <https://huggingface.co/blog/annotated-diffusion>. 3
- [13] Phil Wang. Denoising diffusion probabilistic model, in pytorch. <https://github.com/lucidrains/denoising-diffusion-pytorch>. 3
- [14] Jesse Schell. *The Art of Game Design: A Book of Lenses*. CRC Press, 2008. 1
- [15] Ygor Rebouças Serpa and Maria Andréia Formico Rodrigues. Towards machine-learning assisted asset generation for games: A study on pixel art sprite sheets. In *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, pages 182–191. IEEE, 2019. 1
- [16] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (pcgml). *IEEE Transactions on Games*, 10(3):257–270, 2018. 1

8. Appendix

Supplemental visualizations and tables have been moved to the appendix. The CollaGAN Architecture is shown in Figure 3. Additional image generation visualization is shown in Figure 4, Figure 5, and Figure 5.

The list of tuned hyperparameters for CollaGAN is in Table 4 and for diffusion is in Table 3. Validation curves for CollaGAN are in Table 7 and for diffusion are in Table 8.

Param Name	Diffusion
dropout	(various)
learning_rate	0.00061252
total_steps	240k
lr_decay_steps	120k
batch_size	4
beta_1	0.9
augment	false
beta_s	0.00716085
beta_schedule	cosine
timesteps	255

Param Name	baseline	tuned
dropout	(various)	conservative
learning_rate	0.0001	0.0001
total_steps	240k	240k
lr_decay_steps	120k	120k
batch_size	4	8
beta_1	0.5	0.5
augment	true	true
cycle_mode	forward	forward
capacity	4	8
lambda_l1	100.0	132.6
lambda_l1_back	10.0	5.5
lambda_l1_ssim	10.0	5.9
lambda_l1_domain	10.0	2.3

Table 4: Tuned hyperparameter values for CollaGAN

Table 3: Tuned hyperparameter values for Diffusion

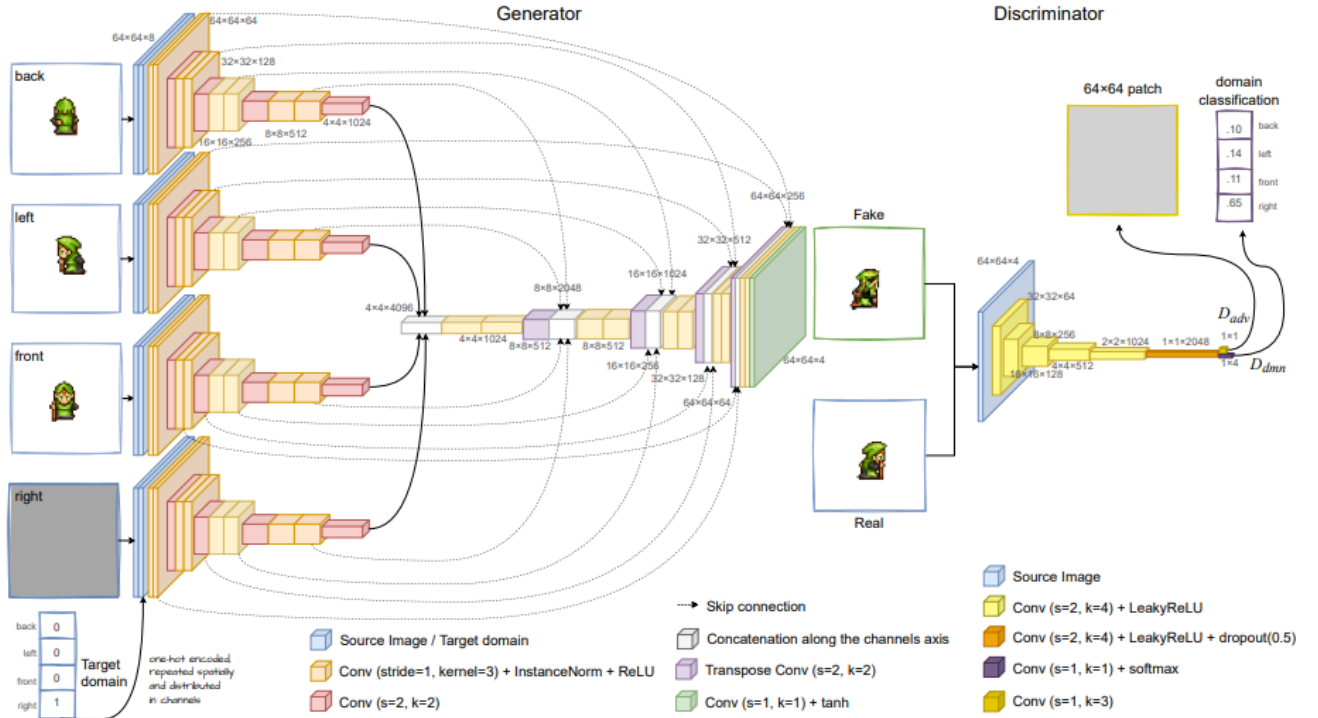


Figure 3: Architecture of CollaGAN [3]

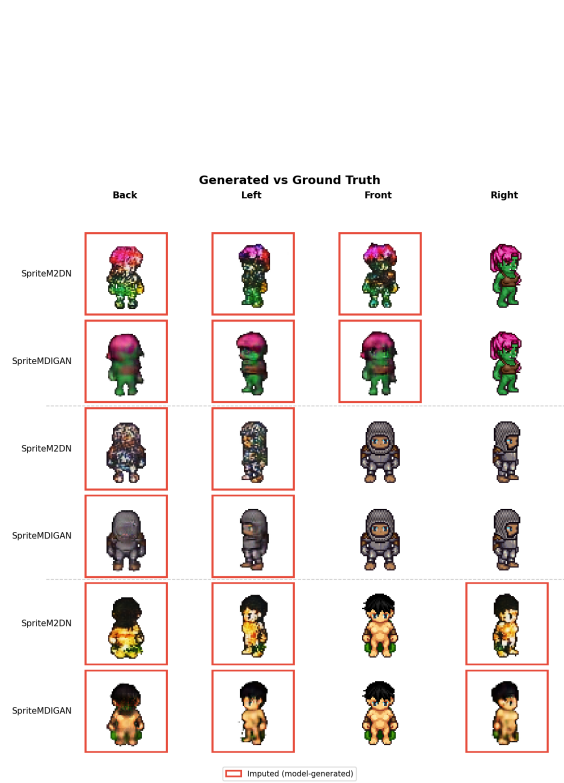


Figure 4: Supplementary image generation comparison for both models. This one also includes some 3-missing scenarios.

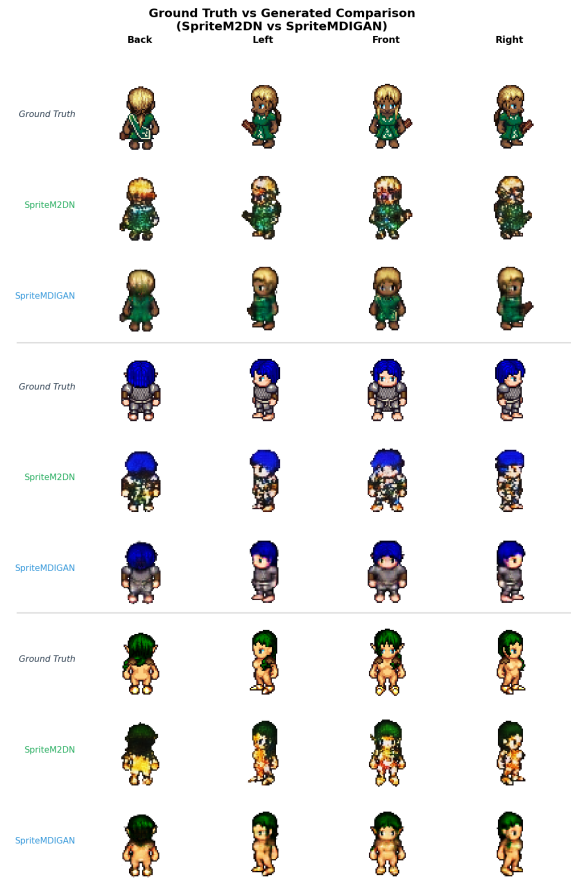


Figure 5: Supplementary image generation comparison for both models. This image compares the full set of poses generated under 1-missing conditions based on the other three poses. Results from both models are shown for the same set of ground truth images.

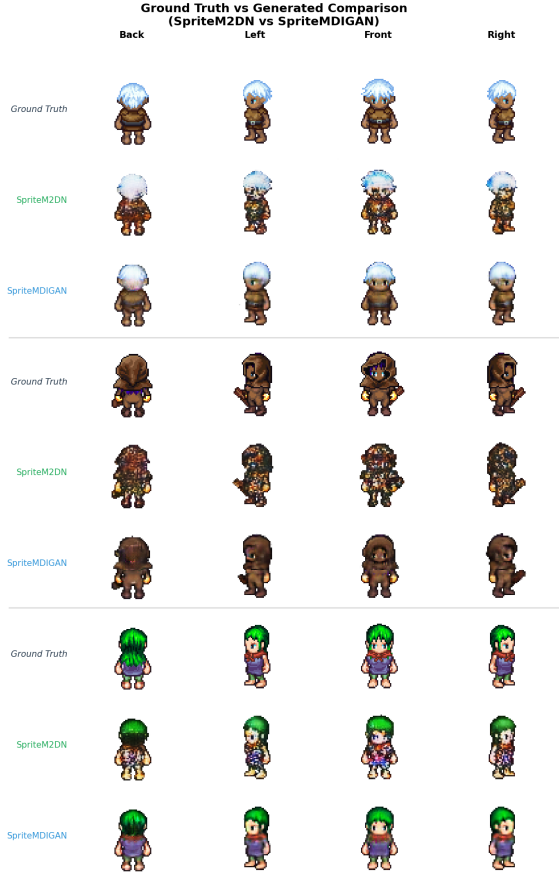


Figure 6: Another example comparing the full set of poses generated under 1-missing conditions based on the other three poses. Results from both models are shown for the same set of ground truth images.

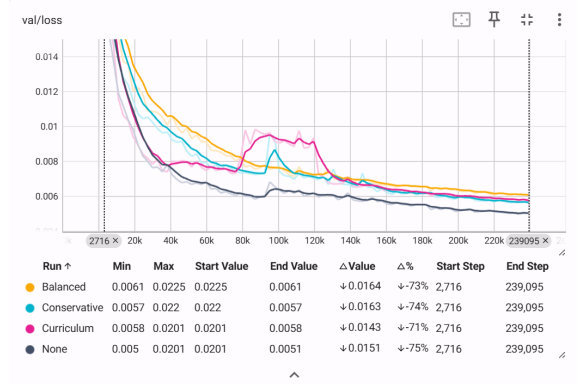


Figure 8: Validation curves for Diffusion under different dropout schemes.

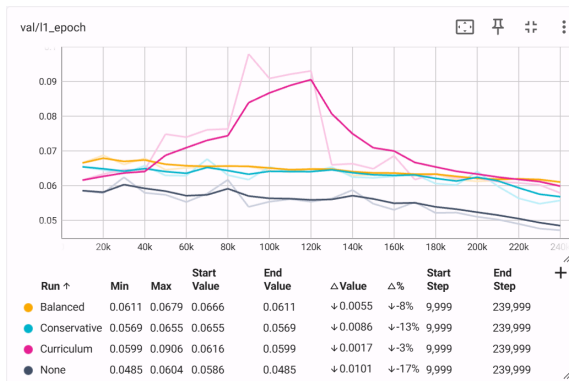


Figure 7: Validation curves for SPrteMDIGAN/CollaGAN under different dropout schemes.